

Challenges & Resolutions

ECS + PostgreSQL Observability Stack — DevOps Engineer Technical Assignment (8Byte.ai)
Koushik M S | Repository: github.com/Koushik-MS-work/ecs-postgres-observability-stack

This document walks through the non-obvious problems encountered while building the four parts of the assignment — infrastructure provisioning, deployment automation, monitoring/logging, and documentation — and how each was resolved. Each entry follows the same structure: the challenge, why it mattered, and the resolution actually implemented in this repository.

Part 1 — Infrastructure Provisioning (Terraform)

1.1. Remote state bootstrap without a chicken-and-egg problem

Challenge: Terraform needs an S3 bucket and DynamoDB lock table to store state remotely, but those resources themselves can't be created by the same Terraform run that depends on them.

Why it mattered: Skipping remote state entirely would make the setup unsafe for a team (no locking, no shared state, risk of concurrent applies corrupting infrastructure) — a bad practice to demonstrate in a DevOps assignment.

Resolution: Treated backend bootstrap as a one-time, out-of-band step: a short block of plain AWS CLI commands (documented in `versions.tf` and the README) that creates the S3 bucket with versioning and the DynamoDB lock table before terraform init is ever run. Everything after that point is fully state-managed.

Relevant files: `terraform/versions.tf`, `README.md`

1.2. Keeping staging and production isolated with one codebase

Challenge: The assignment implies a single infrastructure codebase but two real environments (staging/production) with different risk tolerance — production needs Multi-AZ, deletion protection, and a final snapshot; staging doesn't need any of that cost.

Why it mattered: Duplicating the Terraform code per environment would violate DRY and create drift risk between environments over time.

Resolution: Used a single module set parameterized by `variables.tf` (`environment`, `db_multi_az`, `deletion_protection`, `single_nat_gateway`, etc.) combined with separate partial backend config files (`backend-staging.hcl`, `backend-production.hcl`) so each environment gets its own state file and can be planned/applied independently with `-var="environment=production"` and environment-specific tfvars overrides.

Relevant files: `terraform/variables.tf`, `terraform/backend-*.hcl.example`, `terraform/terraform.tfvars.example`

1.3. Secrets never touching Terraform state or Git

Challenge: RDS requires a master password, and naively passing it as a Terraform variable leaves it readable in plaintext in the state file and risks accidental commits of a `.tfvars` file.

Why it mattered: A leaked DB credential is one of the most common real-world breach vectors, and "secrets in state" is a classic Terraform anti-pattern reviewers specifically look for.

Resolution: Used the `random_password` resource combined with `aws_secretsmanager_secret` / `aws_secretsmanager_secret_version` so the password is generated at apply-time and only ever referenced by ARN afterward — application and CI/CD read it from Secrets Manager at runtime, never from a variable or `tfvars` file. `terraform.tfvars` is also `.gitignore'd` as a second layer of protection.

Relevant files: `terraform/rds.tf`, `.gitignore`

1.4. Least-privilege network path without breaking connectivity

Challenge: It's easy to either over-open security groups (0.0.0.0/0 everywhere, which is insecure) or under-open them (which silently breaks the app-to-database path and is hard to debug from the outside).

Why it mattered: A DB reachable from the public internet, or an SSH port left open on Fargate tasks, would be a red flag in the security review this assignment explicitly asks for.

Resolution: Chained security groups explicitly: ALB SG accepts inbound 443/80 from the internet; app SG only accepts inbound from the ALB SG on the container port; DB SG only accepts inbound from the app SG on 5432. No SSH ingress rule exists anywhere, since Fargate tasks don't need it — the CI/CD pipeline redeploys tasks rather than SSH-ing into them.

Relevant files: `terraform/security_groups.tf`

Part 2 — Deployment Automation (CI/CD)

2.1. Production deploys needed a human gate without a separate pipeline

Challenge: The assignment calls for staging/production deployment logic, but a single linear pipeline that auto-deploys straight to production on every merge to main is risky for a financial-services-adjacent audience like 8Byte.ai.

Why it mattered: Uncontrolled production deploys are a common cause of outages, and "approval-gated production" is a best practice worth demonstrating rather than describing.

Resolution: Used GitHub's built-in Environments feature: `cd.yml` deploys to the staging environment automatically, then targets the production environment for the final job. Because production has one or more required reviewers configured under Settings -> Environments, the workflow run pauses and waits for manual approval before the production deploy step executes — no separate pipeline or custom approval logic needed.

Relevant files: `.github/workflows/cd.yml`

2.2. Testing against a real Postgres instance in CI without provisioning RDS

Challenge: Integration tests need a real PostgreSQL connection, but spinning up actual RDS for every PR run would be slow, costly, and require live AWS credentials in every CI run just to run tests.

Why it mattered: Mocking the database entirely would leave query/migration bugs undetected until deployment — the opposite of what a CI pipeline is for.

Resolution: Used a Postgres service container directly inside the GitHub Actions job (services: postgres:) so integration tests run against a real, ephemeral Postgres instance on localhost within the runner, with zero AWS dependency and no risk to any real database.

Relevant files: `.github/workflows/ci.yml`

2.3. Blocking bad images without blocking every image

Challenge: Failing the pipeline on any vulnerability finding at all would make the pipeline unusable in practice (base images almost always have some low/medium CVEs); never blocking on anything defeats the purpose of scanning.

Why it mattered: The assignment specifically calls out Docker image handling and security as evaluation criteria.

Resolution: Configured Trivy to scan both dependencies (npm audit + Trivy fs scan) and the built image, but only fail the build on CRITICAL severity findings. This keeps the pipeline actionable day-to-day while still hard-blocking on the class of vulnerability that actually matters for a production deploy.

Relevant files: `.github/workflows/ci.yml`, `.github/workflows/cd.yml`

2.4. Long-lived AWS keys in GitHub Secrets

Challenge: The straightforward way to let GitHub Actions deploy to AWS is to store a static AWS access key/secret as a repo secret — but a leaked long-lived key is a standing risk with no automatic expiry.

Why it mattered: Static credentials are one of the most flagged issues in DevOps security reviews, and the assignment explicitly asks for security considerations to be documented.

Resolution: Used OIDC federation instead: GitHub Actions assumes a short-lived IAM role (AWS_DEPLOY_ROLE_ARN / AWS_TERRAFORM_ROLE_ARN, separate roles and scopes for staging vs production) via aws-actions/configure-aws-credentials with no static keys stored anywhere. The deploy role is scoped narrowly to ecs:* on the target cluster/service and ecr:* on the one repository, not broad administrator access.

Relevant files: `.github/workflows/cd.yml`, `SECURITY.md`

Part 3 — Monitoring & Logging

3.1. Two dashboards without duplicating monitoring stacks

Challenge: The assignment asks for at least two dashboards and centralized logging, but a pure-AWS approach (CloudWatch only) and a pure-OSS approach (Prometheus/Grafana only) each have gaps: CloudWatch dashboards aren't runnable locally without an AWS account, while a Prometheus/Grafana-only stack duplicates what ECS Container Insights already gives you for infrastructure metrics.

Why it mattered: A reviewer needs to be able to see the monitoring working without necessarily standing up the full AWS environment first.

Resolution: Provided both, for different purposes: two always-on CloudWatch dashboards (--infrastructure and --application) are provisioned directly by Terraform as part of the real AWS deployment, while a separate docker-compose-based Prometheus + Grafana + postgres-exporter + node-exporter + cAdvisor stack under monitoring/ lets the same metrics story be demoed locally in minutes, with dashboards auto-provisioned from monitoring/grafana/dashboards/ rather than clicked together by hand.

Relevant files: `terraform/monitoring.tf`, `monitoring/docker-compose.monitoring.yml`, `monitoring/grafana/dashboards/`

3.2. Centralizing logs from three different sources

Challenge: Application logs (container stdout), network/infra logs (VPC-level), and database logs (RDS) all originate differently and none of them land in one place by default.

Why it mattered: "Centralized logging" only counts if an on-call engineer can actually find all three log types without knowing three different consoles to check.

Resolution: Routed all three into CloudWatch Logs deliberately: application logs via the awslogs driver on the ECS task definition to /ecs/-, VPC Flow Logs to their own log group for network-level visibility, and RDS postgresql/upgrade logs exported directly from the RDS instance. ALB access logs go to S3 (their supported destination) rather than CloudWatch, since they're typically queried in bulk rather than tailed live.

Relevant files: terraform/ecs.tf, terraform/vpc.tf, terraform/rds.tf, terraform/alb.tf

3.3. Fluent Bit config that works both locally and inside ECS

Challenge: The local reference stack and the real ECS deployment need log shipping configured differently — ECS uses FireLens sidecars, while local Docker Compose has no such concept.

Why it mattered: Writing two unrelated logging configs would mean maintaining duplicate, drifting logic for the same underlying job.

Resolution: Wrote one Fluent Bit config that ships to stdout by default (so it's inspectable immediately in the local stack) with a commented-out cloudwatch_logs output block ready to enable, and documented that the identical file can be reused as the config for an ECS FireLens sidecar container — one source of truth for the log-shipping logic either way.

Relevant files: monitoring/ (Fluent Bit config), README.md

Part 4 — Documentation & Cross-Cutting

4.1. Documenting cost decisions, not just what was built

Challenge: It's easy to document what each Terraform resource does, but the assignment (and a startup's real constraints) also care about why a particular sizing or topology was chosen over a more expensive alternative.

Why it mattered: A production-style reference stack that only lists resources, without justifying trade-offs, doesn't actually demonstrate DevOps judgment — just Terraform syntax.

Resolution: Added an explicit "Cost optimization measures" section to the README covering the Fargate-over-EC2 choice, db.t4g.micro as a Graviton/burstable default with Multi-AZ opt-in, single NAT gateway by default with a documented switch for production, ECR/S3 lifecycle rules, RDS storage autoscaling, and FARGATE_SPOT as an available (not default) capacity provider for interruption-tolerant workloads.

Relevant files: README.md (Cost optimization measures), terraform/ecs.tf, terraform/rds.tf

4.2. Making the repo runnable standalone without a real container image yet

Challenge: terraform apply for Part 1 needs to succeed on its own, before any CI/CD pipeline has ever run and pushed a real application image to ECR — but the ECS task definition needs some image reference to

be valid.

Why it mattered: Reviewers should be able to run Part 1 in isolation to sanity-check the infrastructure, without first needing Part 2's pipeline to have executed.

Resolution: Defaulted `container_image` to a public placeholder image so the Fargate service comes up cleanly on a fresh terraform apply. The CD pipeline updates the running task definition's image directly via `aws ecs register-task-definition` on each deploy, independent of Terraform's managed default — so Parts 1 and 2 can be exercised and verified independently of each other.

Relevant files: terraform/variables.tf, terraform/ecs.tf, .github/workflows/cd.yml, README.md (Assumptions & scope notes)

4.3. Scoping ACM/HTTPS/Route 53/WAF honestly

Challenge: A fully wired HTTPS listener, custom domain, and WAF would make the stack look more "finished", but all three genuinely require a domain name and certificate that only the deployer can supply — faking them with dummy values would make terraform apply fail for anyone else running this repo.

Why it mattered: Silently omitting them (without saying so) would look like an oversight rather than a scoping decision, which matters for the "architectural decisions" part of the documentation requirement.

Resolution: Left ACM/HTTPS, Route 53, and WAF stubbed/commented in the relevant `.tf` files with inline comments explaining what to uncomment and supply, and called this out explicitly as a scope note in the README rather than leaving it undocumented.

Relevant files: terraform/alb.tf, README.md (Assumptions & scope notes)

Overall, the recurring theme across all four parts was choosing the option that stays honest under review — least-privilege over convenience, documented trade-offs over silent omissions, and a stack that can be verified in pieces rather than only as one all-or-nothing deployment.